

Linux Embebido – Módulo 6

UART real, el bus `serdev` y el integrador con el ESP32-S3

Ing. Marín Ariel · Fabrizio Carlassara (BIDOC) ·

Pablo Gómez (BIDOC) · Alexis Hernández de la Cruz (BIS)

Técnicas Digitales III

Laboratorio de Sistemas Embebidos

Universidad Tecnológica Nacional

Facultad Regional Avellaneda

Departamento de Ingeniería Electrónica

Índice

1. Objetivos del módulo	3
2. Punto de partida: del binding manual al automático	4
3. El overlay real: un UART secundario dedicado	4
3.1. Habilitar el UART	4
3.2. <code>pinctrl</code> : asignar los pines a mano	5
3.3. Nuestro nodo: el ESP32-S3 como hijo del UART	6
4. El bus serdev	8
4.1. Por qué no alcanza con <code>/dev/ttyAMAX</code>	8
4.2. <code>struct serdev_device_driver</code> y el ciclo de vida	9
5. El protocolo con el ESP32-S3	10
6. <code>read()</code> como productor-consumidor	11
7. <code>ioctl(ESP32_GET, ...)</code> : sincronismo con <code>completion</code>	12
8. Los comandos <code>ioctl</code> y el reparto completo	14
9. Práctica: el driver completo	15
9.1. Montaje físico	15
9.2. Overlay	16
9.3. El driver: <code>esp32_link.c</code>	16
9.4. El programa de usuario multi-hilo: <code>esp32_ctl.c</code>	17
9.5. Compilación, carga y prueba	19
10. Cierre conceptual	20

Presentación del módulo

Este es el último módulo del bloque, y fusiona lo que originalmente iban a ser dos clases separadas: terminar de ver drivers de dispositivo con un caso real (un UART) y hacer el integrador con la placa ESP32-S3 corriendo FreeRTOS. Hasta acá, los Módulos 4 y 5 construyeron `/dev/ledbtn` sobre hardware inventado para el curso (un LED y un botón), con un truco deliberado en `module_init`: buscar el nodo del Device Tree a mano con `of_find_compatible_node`. Ningún driver real hace eso. Acá se reemplaza esa búsqueda manual por el mecanismo real del kernel – un driver se **registra** y el kernel se lo entrega solo cuando aparece hardware compatible –, aplicado a un periférico real: un UART secundario de la Raspberry Pi, con el que se va a hablar un protocolo propio con el ESP32-S3. Como el UART no es un GPIO cualquiera sino un periférico que ya tiene su propio driver de mainline, aparece un actor nuevo: el bus `serdev`, que le permite a **otro** driver de kernel (el nuestro) ser dueño del puerto y hablar el protocolo. Y como la respuesta del ESP32-S3 llega de forma completamente asíncrona, aparece el problema de sincronización que ya se resolvía en espacio de usuario en el Módulo 3 (`pthread_mutex_t`/`pthread_cond_t`) – ahora hay que resolverlo en espacio de kernel.

Nota

Tres puntos de este driver donde la documentación y lo que realmente compila/anda en la Pi no coinciden:

- `receive_buf` tiene que devolver `size_t`, no `int` – la firma vieja ni compila.
- **La Pi 4 y la Pi 5 no comparten mecanismo de `pinctrl`**: la Pi 5 maneja el GPIO con un chip aparte (RP1, por PCIe), con su propio esquema de pines – y encima UART3 queda enrutado a otros pines físicos (Sección 3.3).
- `serdev_device_write()` exige un `write_wakeup` registrado para aceptar un timeout distinto de cero – sin él, falla siempre con `-EINVAL`, incluso aunque el resto del driver esté perfecto (Sección 4.2).

Misma disciplina que ya destapó, en el Módulo 4, que `gpiod_get_from_of_node` no andaba y había que usar `fwnode_gpiod_get_index`: lo que compiló y anduvo en la Pi real es la referencia, no lo que parece correcto por documentación.

1. Objetivos del módulo

Al finalizar este módulo, el alumno debería poder:

- Explicar por qué buscar un nodo a mano en `module_init` (Módulos 4-5) no es el patrón real, y qué mecanismo del kernel lo reemplaza (`of_match_table` + `probe()`).
- Habilitar un UART secundario dedicado en la Raspberry Pi con un overlay, entendiendo qué rol cumple `pinctrl` en asignar los pines físicos a la función del periférico.

- Describir hardware conectado a un UART como un nodo hijo de ese UART en el Device Tree.
- Explicar qué problema resuelve el bus `serdev` y registrar un `serdev_device_driver` que se bindea automáticamente a ese nodo.
- Diseñar un protocolo de mensajes de texto simple sobre ese UART.
- Implementar `read()` como un productor-consumidor en espacio de kernel (`kfifo` + `wait queue`).
- Implementar un `ioctl` que manda un mensaje y se bloquea hasta la respuesta (`struct completion` + `mutex` + `timeout`), y otro que no espera nada.
- Escribir un programa de usuario multi-hilo (`pthread`) que combine ambas interfaces.

2. Punto de partida: del binding manual al automático

En `ledbtn_init` (Módulos 4-5), el propio módulo llamaba a `of_find_compatible_node` para *salir a buscar* su nodo en el árbol. Funciona, pero no es lo que hace un driver real: si el hardware todavía no está presente cuando el módulo carga, la búsqueda simplemente falla, y no hay forma de que el nodo "avise" más tarde que apareció.

El patrón real es al revés: el driver **se registra** ante un subsistema del kernel ("estoy acá, y sé manejar hardware con este `compatible`"), y es el kernel el que, cada vez que aparece un nodo con ese `compatible` – al arrancar, o al aplicar un overlay en caliente –, llama a la función `probe()` del driver. Esto es `of_match_table`: una tabla de strings `compatible` que el driver declara, y que el kernel compara contra cada nodo del árbol.

Este mecanismo es el mismo, estructuralmente, tanto para un `platform_driver` genérico como para el bus `serdev` que se usa en este módulo (Sección 4): en los dos casos hay un `of_match_table` y una función `probe()` que el kernel invoca solo, sin que el driver tenga que buscar nada.

3. El overlay real: un UART secundario dedicado

3.1. Habilitar el UART

La Raspberry Pi (BCM2711, la familia de la Pi 4) tiene varios UART secundarios además del que usa la consola serie del sistema, cada uno deshabilitado por defecto en el árbol base. El elegido para este módulo es **UART3**, en GPIO4 (TXD3, pin físico 7) y GPIO5 (RXD3, pin físico 29) – pines de uso general, sin ningún rol especial reservado (a diferencia de, por ejemplo, GPIO0/GPIO1, que la Pi usa para detectar HATs con EEPROM, o GPIO14/GPIO15, la consola serie primaria).

El nodo `uart3` en el árbol base ya existe, pero deshabilitado:

```
uart3: serial@... {
    compatible = "arm,pl011", "arm,primecell";
    /* ... reg, interrupts, clocks ... */
```

```
status = "disabled";
};
```

arm,pl011 es el mismo controlador serie ARM que usan varios UART de la Pi – ya tiene su driver en el kernel mainline. La Raspberry Pi Foundation distribuye un overlay oficial que lo habilita (`sudo dtoverlay uart3`), pero – mismo espíritu que el Módulo 4, que escribió su propio overlay para el LED/botón en vez de depender de uno ajeno – acá se escribe el propio, completo, sin depender del oficial. La diferencia con el Módulo 4 es que ahí se pedía un GPIO de uso general (`gpiod_get_from_of_node/fwnode_gpiod_get_index` ya reciben el pin listo para usar); acá hace falta, por primera vez en el curso, tocar el *pin multiplexer* del SoC a mano.

Nota

El overlay oficial (`uart3-overlay.dts`) es tan corto que no hace falta más que esto:

```
fragment@0 {
    target = <&uart3>;
    __overlay__ {
        status = "okay";
    };
};
```

Y nada más – porque el mapeo de GPIO4/GPIO5 a la función UART3 ya está descrito en algún lugar del árbol base; el overlay solo despierta el nodo. Acá se prefiere no depender de esa asignación implícita y describirla explícitamente en el propio overlay del curso.

3.2. pinctrl: asignar los pines a mano

`pinctrl` es el subsistema del kernel que decide qué **función** cumple cada pin físico del SoC – GPIO de uso general, o una de varias funciones alternativas (I2C, SPI, UART, etc.), según el *pin multiplexer* del chip. El Módulo 4 dejó esto explícitamente afuera (“sin `probe()/platform_driver/pinctrl`”): acá aparece por primera vez.

En el BCM2711, cada GPIO tiene un registro GPFSELn de 3 bits que elige su función:

Valor	Función
000	entrada
001	salida
100	ALT0
101	ALT1
110	ALT2
111	ALT3
011	ALT4
010	ALT5

Nótese que las funciones alternativas **no** están en orden binario (ALT4 es 011 = decimal 3, no 4) – una fuente común de errores al escribir un `brcm, function` a mano. La tabla de asignación de funciones alternativas del datasheet del BCM2711 dice, para los pines elegidos, que GPIO4 en ALT4 es TXD3 y GPIO5 en ALT4 es RXD3 [7] – exactamente el UART3 que se quiere usar.

Un grupo de `pinctrl` propio se describe como un nodo hijo del controlador de GPIO (`&gpio`), con `brcm,pins` (qué GPIO), `brcm,function` (qué función alternativa, con la codificación de la tabla de arriba) y `brcm,pull` (resistencia de pull-up/pull-down; 0 = ninguna, 2 = pull-up):

```
fragment@0 {
    target = <&gpio>;
    __overlay__ {
        esp32_link_pins: esp32_link_pins {
            brcm,pins = <4 5>;
            brcm,function = <3>; /* ALT4: GPIO4=TXD3, GPIO5=RXD3 */
            brcm,pull = <0 2>; /* TX sin pull, RX con pull-up */
        };
    };
};
```

Nota

El RX lleva pull-up (2) por la misma razón que ya se vio con el botón del Módulo 4: la línea tiene que quedar en un nivel definido (alto, el reposo típico de un UART) cuando el ESP32-S3 todavía no está transmitiendo nada. El TX no necesita pull porque siempre lo maneja activamente el propio controlador PL011.

3.3. Nuestro nodo: el ESP32-S3 como hijo del UART

El grupo de `pinctrl` de arriba todavía no está conectado a ningún periférico. Falta un segundo fragmento que tome `&uart3`, lo apunte a ese grupo con `pinctrl-names/pinctrl-0`, lo habilite, y agregue al ESP32-S3 como nodo **hijo** – que es donde el bus `serdev` (Sección 4) espera encontrar a sus dispositivos:

```
/dts-v1/;
/plugin/;

/ {
    compatible = "brcm,bcm2711";

    fragment@0 {
        target = <&gpio>;
        __overlay__ {
            esp32_link_pins: esp32_link_pins {
                brcm,pins = <4 5>;
                brcm,function = <3>; /* ALT4: GPIO4=TXD3, GPIO5=RXD3 */
            };
        };
    };
};
```

```

        brcm,pull = <0 2>; /* TX sin pull, RX con pull-up */
    };
};

fragment@1 {
    target = <&uart3>;
    __overlay__ {
        pinctrl-names = "default";
        pinctrl-0 = <&esp32_link_pins>;
        status = "okay";

        esp32_link: esp32-link {
            compatible = "td3,esp32-link";
            current-speed = <115200>;
            status = "okay";
        };
    };
};
};

```

Un solo overlay, propio, sin depender del de la Raspberry Pi Foundation: fragmento 0 declara el `pinctrl`, fragmento 1 lo conecta a `uart3`, lo enciende, y cuelga el nodo del ESP32-S3. `current-speed` es una propiedad estándar (no inventada para este curso) que describe el baudrate esperado de un dispositivo serie hijo de un UART; el driver la lee en `probe()` en vez de tener el baudrate escrito a fuego en el código C (Sección 4.2).

Nota

Este overlay es específico de la Pi 4 (BCM2711). La Pi 5 (BCM2712) maneja el GPIO/UART con un chip aparte, el **RP1** (conectado por PCIe), con su propio controlador de pines – la sintaxis `brcm, pins/brcm, function` de arriba no le sirve. En una Pi 5, UART3 queda enrutado a GPIO8 (pin físico 24, TXD3) y GPIO9 (pin físico 21, RXD3) – pines distintos a los de la Pi 4 –, y como todavía no está verificada la codificación numérica del pinmux de RP1, el overlay para Pi 5 no arma el grupo de `pinctrl` a mano: referencia el `uart3_pins` que ya viene correcto en el árbol base de la Pi 5, mismo patrón que el overlay oficial `uart3-pi5-overlay.dts` de Raspberry Pi:

```
/dts-v1/;
/plugin/;

/ {
    compatible = "brcm,bcm2712";

    fragment@0 {
        target = <&uart3>;
        __overlay__ {
            pinctrl-0 = <&uart3_pins>;
            status = "okay";

            esp32_link: esp32-link {
                compatible = "td3,esp32-link";
                current-speed = <115200>;
                status = "okay";
            };
        };
    };
};
```

Los dos overlays completos (`esp32-link-pi4-overlay.dts/esp32-link-pi5-overlay.dts`) están en el código fuente de la práctica; el driver (`esp32_link.c`) es exactamente el mismo para las dos placas – lo único que cambia es qué overlay se aplica.

4. El bus serdev

4.1. Por qué no alcanza con `/dev/ttyAMAX`

Con `uart3` habilitado nada más, el kernel expone un `/dev/ttyAMAX` que cualquier proceso de usuario podría abrir con `open/read/write` y hablarle directamente al ESP32-S3 – de hecho, así se puede probar el enlace a mano con una terminal (Sección 9). Pero acá el objetivo es que sea el propio **driver de kernel** el dueño del puerto: el que arma el protocolo, sincroniza pedido y respuesta, y expone `read/write/ioctl` de más alto nivel a espacio de usuario.

Para que un driver de kernel (no un proceso) sea el consumidor exclusivo de un UART, existe el bus `serdev` (*serial device*) – el mismo mecanismo que usan, por ejemplo, los drivers de chips Bluetooth conectados por UART.

4.2. struct serdev_device_driver y el ciclo de vida

Del lado del registro, es el mismo mecanismo de `of_match_table` + `probe()` de la Sección 2, pero sobre el bus `serdev` en vez de sobre `platform`:

```
static const struct of_device_id esp32_of_match[] = {
    { .compatible = "td3,esp32-link" },
    { }
};
MODULE_DEVICE_TABLE(of, esp32_of_match);

static struct serdev_device_driver esp32_driver = {
    .probe = esp32_probe,
    .remove = esp32_remove,
    .driver = {
        .name = "esp32_link",
        .of_match_table = esp32_of_match,
    },
};
module_serdev_device_driver(esp32_driver);
```

`module_serdev_device_driver` es el equivalente, para este bus, de `module_platform_driver`: genera los `module_init/module_exit` que registran y desregistran el driver, sin escribirlos a mano.

Dentro de `probe()`, el ciclo de vida del lado `serdev` es:

1. `serdev_device_set_client_ops(serdev, &ops)`: registra el callback que procesa los bytes entrantes (`receive_buf`, Sección 6) – **antes** de abrir el puerto.
2. `serdev_device_open(serdev)`: abre el puerto.
3. `of_property_read_u32()` seguido de `serdev_device_set_baudrate()`: en vez de un baudrate fijo en el código, se lee la propiedad `current-speed` que declaró el overlay (Sección 3.3).
4. `serdev_device_set_flow_control(serdev, false)`: sin control de flujo por hardware, coherente con no haber pedido `ctsrts` en el overlay.

Nota

El registro de `esp32_serdev_ops` necesita un segundo campo además de `receive_buf`: `write_wakeup`. `serdev_device_write()` (Sección 8) lo usa para enterarse de cuándo se liberó espacio en el buffer de transmisión mientras espera – y **exige** que esté registrado si se lo llama con un `timeout` distinto de cero: sin él, la llamada rechaza de entrada con `-EINVAL`, sin transmitir un solo byte, aunque el resto del driver esté perfecto. Alcanza con un callback vacío – no hace falta que haga nada, solo que exista:

```
static void esp32_write_wakeup(struct serdev_device *serdev)
{
}

static const struct serdev_device_ops esp32_serdev_ops = {
    .receive_buf = esp32_receive_buf,
    .write_wakeup = esp32_write_wakeup,
};
```

Este error es fácil de pasar por alto porque no se manifiesta ni en la compilación ni en `probe()`: el driver carga bien, el nodo se crea bien, y recién falla adentro de `write()/ioctl`. Por eso importa el segundo hábito, independiente de este bug puntual: nunca ignorar lo que devuelve `serdev_device_write()` (Sección 8) – puede fallar, o escribir menos bytes de los pedidos, sin que sea un error negativo.

5. El protocolo con el ESP32-S3

El protocolo es texto plano, una línea por mensaje, terminada en `\n` – simple de generar e implementar en el firmware FreeRTOS del ESP32-S3 (fuera del alcance de este apunte: cada alumno tiene el suyo, y va a ser distinto), y fácil de probar a mano desde una terminal serie. Tres comandos:

- `set VARIABLE VALOR\n` – le pide al ESP32-S3 que fije una variable propia (por ejemplo, un setpoint). Sin respuesta esperada.
- `get VARIABLE\n` – pide el valor actual de una variable. El ESP32-S3 responde `VARIABLE=VALOR\n`.
- `exec ACCION\n` – le pide que ejecute una acción puntual (por ejemplo, un toggle). Sin respuesta esperada.

Nota

El firmware del ESP32-S3 no se provee en este apunte – cada alumno tiene su propia implementación en FreeRTOS, distinta de la de sus compañeros. Lo que sí es parte del contrato que este apunte fija, y que el firmware de cada uno tiene que respetar, es el formato exacto de las tres líneas de arriba, en particular la de la respuesta a `get`.

6. read() como productor-consumidor

Cualquier línea que llegue del ESP32-S3 – sea la respuesta a un `get`, o algo que el firmware decida mandar por su cuenta – tiene que quedar disponible para quien esté leyendo el dispositivo con `read()` de forma cruda (por ejemplo, con `cat`, o con el hilo monitor de la Sección 9.4). Como `receive_buf()` corre en un contexto que no tiene nada que ver con cuándo un proceso llamó a `read()`, hace falta el mismo patrón productor-consumidor de la Práctica 2 del Módulo 3 (buffer circular + mutex + variables de condición) – pero en espacio de kernel, con `kfifo` (un buffer circular ya implementado por el kernel) y una wait queue en vez de `pthread_cond_t`:

```
static struct kfifo rx_fifo;
static DECLARE_WAIT_QUEUE_HEAD(rx_wait);

static size_t esp32_receive_buf(struct serdev_device *serdev,
                               const unsigned char *data, size_t count)
{
    size_t i;

    for (i = 0; i < count; i++) {
        if (linelen < sizeof(linebuf) - 1)
            linebuf[linelen++] = data[i];

        if (data[i] == '\n') {
            kfifo_in(&rx_fifo, linebuf, linelen);
            wake_up_interruptible(&rx_wait);

            if (esperando_get) {
                linebuf[linelen] = '\0';
                strcpy(get_response, linebuf, sizeof(get_response));
                complete(&get_done);
            }

            linelen = 0;
        }
    }

    return count; /* consumimos todo lo que nos dieron */
}

static ssize_t esp32_read(struct file *file, char __user *buf,
                        size_t len, loff_t *off)
{
    unsigned int copiados;
    int ret;

    if (kfifo_is_empty(&rx_fifo)) {
        if (file->f_flags & O_NONBLOCK)
            return -EAGAIN;
    }
}
```

```

    if (wait_event_interruptible(rx_wait, !kfifo_is_empty(&rx_fifo)))
        return -ERESTARTSYS;
}

ret = kfifo_to_user(&rx_fifo, buf, len, &copiados);
if (ret)
    return ret;

return copiados;
}

```

`linebuf/linelen` van ensamblando bytes hasta encontrar un `\n` – porque `receive_buf()` puede recibir los bytes de una línea repartidos en varias llamadas, o varias líneas juntas en una sola llamada, según cómo le convenga entregarlos al controlador de UART. `esp32_read` no usa `*off`: igual que `write()` en el Módulo 4, no tiene sentido posicional acá, el `kfifo` se va consumiendo a medida que se lee.

Nota

Cada línea completa se copia al `kfifo` **siempre**, esté alguien esperando un `get` o no – eso es lo que le permite al hilo monitor de la CLI (Sección 9.4) ver todo lo que llega, no solo las respuestas a los pedidos que él mismo hizo.

7. `ioctl(ESP32_GET, ...)`: sincronismo con completion

El `kfifo` de la sección anterior alcanza para *mirar* lo que llega, pero no sirve para implementar `get` de forma confiable: si dos cosas leen del mismo `kfifo`, la línea que a uno le interesaba se la puede llevar el otro. `ioctl(ESP32_GET, ...)` necesita su propio canal, dedicado, entre “mandé el pedido” y “llegó la respuesta que corresponde a ese pedido puntual”. La Figura 1 resume cómo una misma línea entrante termina en las dos vías sin que se pisen.

```

static struct completion get_done;
static struct mutex get_lock;
static char get_response[MAX_LINE];
static bool esperando_get;

static long esp32_ioctl_get(struct esp32_var __user *argp)
{
    struct esp32_var var;
    char msg[48];
    char *igual;
    long ret;
    int wret;

    if (copy_from_user(&var, argp, sizeof(var)))

```

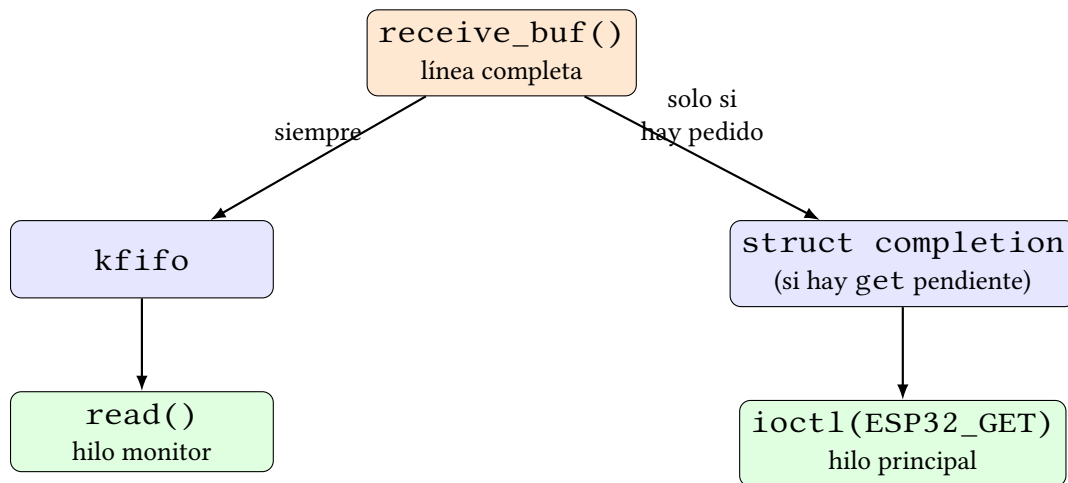


Figura 1: Una misma línea entrante alimenta dos vías independientes: el **kfifo** (para cualquiera que esté mirando con **read()**) y, si hay un **get** en vuelo, también el **completion** de ese pedido puntual. Ninguna de las dos vías le saca la línea a la otra.

```

    return -EFAULT;
    var.nombre[sizeof(var.nombre) - 1] = '\0';

    mutex_lock(&get_lock);
    reinit_completion(&get_done);
    esperando_get = true;

    snprintf(msg, sizeof(msg), "get %s\n", var.nombre);
    wret = serdev_device_write(esp32_serdev, msg, strlen(msg),
        msecs_to_jiffies(100));
    if (wret < 0) {
        esperando_get = false;
        mutex_unlock(&get_lock);
        return wret;
    }

    ret = wait_for_completion_interruptible_timeout(&get_done,
        msecs_to_jiffies(500));
    esperando_get = false;

    if (ret == 0) {
        mutex_unlock(&get_lock);
        return -ETIMEDOUT; /* el ESP32-S3 no contesto a tiempo */
    }
    if (ret < 0) {
        mutex_unlock(&get_lock);
        return ret; /* -ERESTARTSYS: la llamada se interrumpe */
    }

    igual = strchr(get_response, '=');

```

```

if (!igual || kstrtoint(igual + 1, 10, &var.valor)) {
    mutex_unlock(&get_lock);
    return -EPROTO; /* la respuesta no tenia el formato esperado */
}
mutex_unlock(&get_lock);

return copy_to_user(argp, &var, sizeof(var)) ? -EFAULT : 0;
}

```

El `mutex_lock(&get_lock)` cubre todo el tramo “mandar + esperar”: si dos procesos llaman a `ioctl(ESP32_GET, ...)` al mismo tiempo, el segundo espera a que termine el primero en vez de pisarle la respuesta. `wait_for_completion_interruptible_timeout` devuelve 0 si venció el timeout sin que nadie llamara a `complete()`, un valor negativo si la espera fue interrumpida por una señal, o el tiempo restante (positivo) si `complete()` se disparó a tiempo – de ahí las tres ramas.

8. Los comandos `ioctl` y el reparto completo

Con las piezas de las tres secciones anteriores, así queda repartida la interfaz completa del dispositivo sobre los tres verbos del protocolo:

Verbo	Vía	Por qué ahí
<code>exec ACCION</code>	<code>write()</code> crudo	No hace falta tipar nada ni esperar respuesta – mandar el string tal cual alcanza.
(lo que llegue)	<code>read()</code>	Bloqueante de verdad (Sección 6) – vía cruda de monitoreo, tipo <code>cat</code> sobre una terminal serie.
<code>set VAR VALOR</code>	<code>ioctl</code> (<code>ESP32_SET</code>) – <code>_IOW</code>	Argumentos tipados (<code>struct esp32_var</code>), <i>fire-and-forget</i> : arma la línea, la manda, no espera nada.
<code>get VAR</code>	<code>ioctl</code> (<code>ESP32_GET</code>) – <code>_IOWR</code>	No es <code>_IOR</code> : hay que mandar el nombre de la variable y recibir el valor en la misma llamada. Bloqueante, con timeout (Sección 7).

El header compartido entre el driver y el programa de usuario:

```

#ifndef _ESP32_LINK_H_
#define _ESP32_LINK_H_

#include <linux/ioctl.h>

#define ESP32_VAR_NAME_MAX 16

```

```

struct esp32_var {
    char nombre[ESP32_VAR_NAME_MAX];
    int valor;
};

#define ESP32_MAGIC 'y'

#define ESP32_SET _IOW (ESP32_MAGIC, 1, struct esp32_var)
#define ESP32_GET _IOWR(ESP32_MAGIC, 2, struct esp32_var)

#endif

```

esp32_ioctl, uniendo todo:

```

static long esp32_ioctl(struct file *file, unsigned int cmd, unsigned
    long arg)
{
    struct esp32_var __user *argp = (struct esp32_var __user *)arg;
    struct esp32_var var;
    char msg[48];
    int wret;

    switch (cmd) {
    case ESP32_SET:
        if (copy_from_user(&var, argp, sizeof(var)))
            return -EFAULT;
        var.nombre[sizeof(var.nombre) - 1] = '\\0';
        snprintf(msg, sizeof(msg), "set %s %d\\n", var.nombre, var.valor);
        wret = serdev_device_write(esp32_serdev, msg, strlen(msg),
            msec_to_jiffies(100));
        return wret < 0 ? wret : 0;

    case ESP32_GET:
        return esp32_ioctl_get(argp);

    default:
        return -ENOTTY;
    }
}

```

9. Práctica: el driver completo

9.1. Montaje físico

- **Pi 4** – GPIO4 (TXD3, pin físico 7) → RX del ESP32-S3; GPIO5 (RXD3, pin físico 29) ← TX del ESP32-S3.
- **Pi 5** – GPIO8 (TXD3, pin físico 24) → RX del ESP32-S3; GPIO9 (RXD3, pin físico 21) ←

TX del ESP32-S3 – distintos a los de la Pi 4 (Sección 3.3).

- GND común entre las dos placas.
- Ambas trabajan a 3.3 V: conexión directa, sin traductor de niveles.

9.2. Overlay

Un solo overlay propio (Sección 3.3), sin depender del oficial de la Raspberry Pi Foundation – `esp32-link-pi4-overlay.dts` o `esp32-link-pi5-overlay.dts` según la placa:

```
dtc -@ -I dts -O dtb -o esp32-link.dtbo esp32-link-piN-overlay.dts
sudo dtoverlay -d . esp32-link
```

Nota

Si `/dev/esp32link` quedó apuntando a algo raro de una prueba anterior (por ejemplo, `echo ... >/dev/esp32link` corrido en algún momento en que el módulo no estaba cargado), puede haber quedado como un **archivo común** en vez del nodo de dispositivo – `udev` no pisa un archivo ya existente al crear el nodo real. Se nota con `ls -l /dev/esp32link`: un nodo de carácter de verdad arranca con `c` y muestra un mayor/minor (`crw— ... 509, 0 ...`); un archivo común arranca con `-` y muestra un tamaño en bytes. Si pasa, `sudo rm -f /dev/esp32link` y volver a cargar el módulo.

9.3. El driver: `esp32_link.c`

El registro del character device (`alloc_chrdev_region`, `cdev_init/ cdev_add`, `class_create/device_create`) es exactamente el mismo patrón del Módulo 4 – se omite acá por espacio, pero va completo en el archivo entregado junto con este apunte. Lo nuevo de este módulo, ya mostrado en las secciones anteriores, se junta todo en `esp32_probe`:

```
static struct serdev_device *esp32_serdev;

static const struct serdev_device_ops esp32_serdev_ops = {
    .receive_buf = esp32_receive_buf,
    .write_wakeup = esp32_write_wakeup, /* obligatorio para timeout != 0,
        Seccion 4.2 */
};

static int esp32_probe(struct serdev_device *serdev)
{
    u32 baudrate = 115200;
    int ret;
```

```

esp32_serdev = serdev;
serdev_device_set_client_ops(serdev, &esp32_serdev_ops);

ret = serdev_device_open(serdev);
if (ret)
    return ret;

of_property_read_u32(serdev->dev.of_node, "current-speed", &baudrate);
serdev_device_set_baudrate(serdev, baudrate);
serdev_device_set_flow_control(serdev, false);

ret = kfifo_alloc(&rx_fifo, 256, GFP_KERNEL);
if (ret) {
    serdev_device_close(serdev);
    return ret;
}

init_completion(&get_done);
mutex_init(&get_lock);

/* alloc_chrdev_region / cdev_init / cdev_add / class_create /
   device_create
   -- igual que en el Modulo 4, omitido acá */

dev_info(&serdev->dev, "esp32-link: listo, baudrate=%u\n", baudrate);
return 0;
}

static void esp32_remove(struct serdev_device *serdev)
{
    /* deshacer el registro del character device -- igual que en el Modulo
       4 */
    kfifo_free(&rx_fifo);
    serdev_device_close(serdev);
}

```

9.4. El programa de usuario multi-hilo: esp32_ctl.c

echo/cat alcanzan para probar exec y para mirar lo que llega, pero no pueden invocar `ioctl()`. El programa de control, a diferencia de `ledbtn_ctl.c` del Módulo 5, tiene **dos hilos**:

- Un hilo **monitor**, en loop bloqueante de `read()`, que imprime cada línea que llega – respuestas a `get`, o cualquier cosa que el firmware decida mandar por su cuenta.
- El hilo **principal**, con un menú interactivo (`set/get/ exec`) que hace `write()/ioctl()` según lo que carga el usuario.

```

#include <stdio.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>
#include <pthread.h>
#include <sys/ioctl.h>
#include "esp32_link.h"

#define DEV_PATH "/dev/esp32link"

static int fd;
static pthread_mutex_t consola = PTHREAD_MUTEX_INITIALIZER;

static void *hilo_monitor(void *arg)
{
    char buf[128];
    ssize_t n;

    while ((n = read(fd, buf, sizeof(buf) - 1)) > 0) {
        buf[n] = '\0';
        pthread_mutex_lock(&consola);
        printf("[esp32] %s", buf);
        pthread_mutex_unlock(&consola);
    }
    return NULL;
}

int main(void)
{
    pthread_t monitor;
    struct esp32_var var;
    char linea[64], accion[16], nombre[ESP32_VAR_NAME_MAX];
    int valor;

    fd = open(DEV_PATH, O_RDWR);
    if (fd < 0) {
        perror("open");
        return 1;
    }

    pthread_create(&monitor, NULL, hilo_monitor, NULL);

    while (fgets(linea, sizeof(linea), stdin)) {
        if (sscanf(linea, "set %15s %d", nombre, &valor) == 2) {
            strncpy(var.nombre, nombre, sizeof(var.nombre));
            var.valor = valor;
            ioctl(fd, ESP32_SET, &var);
        }
    }
}

```

```

    } else if (sscanf(linea, "get %15s", nombre) == 1) {
        strncpy(var.nombre, nombre, sizeof(var.nombre));
        if (ioctl(fd, ESP32_GET, &var) == 0) {
            pthread_mutex_lock(&consola);
            printf("%s = %d\n", var.nombre, var.valor);
            pthread_mutex_unlock(&consola);
        } else {
            perror("ioctl ESP32_GET");
        }
    }

    } else if (sscanf(linea, "exec %15s", accion) == 1) {
        snprintf(linea, sizeof(linea), "exec %s\n", accion);
        write(fd, linea, strlen(linea));
    }
}

close(fd);
return 0;
}

```

Nota

El mutex `consola` no protege ningún dato del protocolo – protege la salida por pantalla, que sí es un recurso compartido genuino entre los dos hilos (el monitor imprimiendo lo que llega, el principal imprimiendo el resultado de un `get`). Sin él, dos `printf` de hilos distintos podrían entrelazar sus caracteres en la terminal. Mismo principio del Módulo 3: todo acceso a un dato compartido necesita el mismo mutex de los dos lados.

9.5. Compilación, carga y prueba

```

# Compilar y cargar el driver
make
sudo insmod esp32_link.ko

# Compilar el programa de control
gcc -Wall -pthread -o esp32_ctl esp32_ctl.c

# Probar exec y el monitor con herramientas genericas
echo "exec toggle" | sudo tee /dev/esp32link
cat /dev/esp32link &

# Usar la CLI completa
sudo ./esp32_ctl
set temp_objetivo 25
get temp

```

```
exec toggle
```

Nota

Si el driver `ledbtn` del Módulo 4/5 sigue cargado, `class_create("td3")` va a fallar acá porque ese nombre de clase ya está tomado en `sysfs` – hace falta `sudo rmmod ledbtn` antes de cargar `esp32_link`, o usar un nombre de clase distinto.

10. Cierre conceptual

Este módulo cierra el recorrido completo del bloque. Los puntos clave:

- Un driver real se registra y espera a que el kernel lo llame (`of_match_table + probe()`) – no sale a buscar su propio hardware, como hacían los Módulos 4-5 a propósito.
- El bus `serdev` le da a un driver de kernel la propiedad exclusiva de un UART, para hablar un protocolo propio en vez de dejarlo como un `/dev/ttyAMAX` genérico.
- `read()` puede ser, del lado del kernel, exactamente el mismo patrón productor-consumidor del Módulo 3 – `kfifo` y `wait queues` en vez de `pthread_cond_t`.
- Cuando hace falta correlacionar un pedido con su respuesta de forma confiable, `ioctl` con `struct completion + mutex + timeout` hace lo que un `read()` genérico no puede garantizar.
- La misma disciplina de sincronización aparece en los dos extremos del curso: `mutex` y variables de condición en espacio de usuario (Módulo 3), y sus equivalentes de kernel acá.

Con esto, el bloque completo recorrió el camino de un módulo de kernel que solo escribía en `dmesg` (Módulo 2) a un driver real, con bus dedicado, protocolo propio, sincronismo en espacio de kernel, y una aplicación de usuario multi-hilo que lo orquesta todo – comunicándose con una placa FreeRTOS real.

Referencias

[1] Vasquez, F. y Simmonds, C. (2021). *Mastering Embedded Linux Programming*, 3.^a edición. Packt Publishing.

- Cap. 3, sección de overlays de Device Tree.
- Cap. 11, *Interfacing with Device Drivers*.
- Cap. 12, *Prototyping with Breakout Boards* – metodología de mapear hardware real a Device Tree, adaptada acá de un periférico SPI/I2C a un UART.

- [2] Corbet, J., Rubini, A. y Kroah-Hartman, G. (2005). *Linux Device Drivers*, 3.^a edición. O'Reilly Media. Disponible libremente en <https://lwn.net/Kernel/LDD3/>. Cap. 6, *Advanced Char Driver Operations – ioctl* y bloqueo de procesos, ya citado en el Módulo 5.
- [3] The Linux Kernel development community. *Serial Device Bus*. <https://docs.kernel.org/driver-api/serial/serdev.html>. Referencia de `struct serdev_device_driver`, `serdev_device_open/ close`, `serdev_device_set_baudrate`, `serdev_device_write`, y el callback `receive_buf` de `struct serdev_device_ops`.
- [4] The Linux Kernel development community. *kfifo – generic FIFO implementation*. Documentado en `include/linux/kfifo.h` del código fuente del kernel. `kfifo_alloc`, `kfifo_in`, `kfifo_to_user`, `kfifo_is_empty`.
- [5] The Linux Kernel development community. *Completions – "wait for completion" barrier APIs*. <https://docs.kernel.org/scheduler/completion.html>. `struct completion`, `init_completion/reinit_completion`, `complete`, `wait_for_completion_interruptible_timeout`.
- [6] Raspberry Pi Ltd. Código fuente de los overlays de Device Tree, incluido `uart2-overlay.dts`/`uart3-overlay.dts`. <https://github.com/raspberrypi/linux/tree/rpi-6.6.y/arch/arm/boot/dts/overlays>.
- [7] Raspberry Pi Ltd. *BCM2711 ARM Peripherals*. <https://datasheets.raspberrypi.com/bcm2711/bcm2711-peripherals.pdf>. Tabla 94 (*GPIO Pins Alternative Function Assignment*): asignación de GPIO4 como TXD3 y GPIO5 como RXD3 en la función alternativa ALT4. Tabla 64 (*GPIO Alternate function select register 0*): codificación de 3 bits de GPFSELn, que explica por qué ALT4 vale 3 y no 4.
- [8] Bootlin. *Elixir Cross Referencer – Linux source code*. <https://elixir.bootlin.com/linux/latest/source>. Para verificar, contra la versión de kernel real de la Pi en uso, las firmas exactas de `serdev_*` y `kfifo_*` antes de dar esta clase.